

VOGON POETRY

Revisiting data engineering concepts with an eye for maximizing
value in an AI-driven value chain

SHAURYA AGARWAL

ABSTRACT

...As plurdled gabbleblotchits on a lurgid bee...

AI is fast becoming the core value engine. The differentiator is no longer the model, but the ability to address determinism, correctness, latency and unit costs. Data Engineering must be looked at from this lens.

In this fast evolving ecosystem, the expert-novice data engineer gap cannot be addressed by building tool fluency, but instead building a high quality understanding of a pedagogy of core concepts - a core set of transferrable mental models that unlock analysis and prediction of behaviour across tools and systems - prevalent and emergent.

This paper organizes these mental models as concepts, classified into "instincts" - invariant ideas that govern a class of decisions, grouped into "families" - that form the general heuristic idea in data engineering. Honestly, when read together - these sound like a poem that would make Prostetnic Vogon Jeltz slurp his axlegrurts hagrilly. Hence the name.

The families function as a dependency structure rather than a taxonomy. A decision in one family, such as file layout under "storage as a substrate," propagates predictable consequences in others, such as scan cost under "mechanical sympathy" and shuffle volume under "distribution."

The author claims that familiarity with these concepts first, then attaching tools to them, yields faster transfer to unfamiliar systems than tool-first instruction, because these mental models largely act as invariants and remain stable while interfaces change. This paper presents the map, the concept graph, and the reasoning each instinct supports, with banking and reinsurance as the primary worked domains.

By order of Galactic Hyperspace Planning Council,
Recitation shall now begin, without mercy.
Resistance is useless.

Family A, Mechanical Sympathy

Physical form dominates performance
Movement is the cost
Batch beats tuple-at-a-time

Family B, Do Less And Prove It

Declare what; let the planner choose how
The fastest work is skipped work
Approximate on purpose

Family C, Distribution

Placement decides what is cheap

Family D, State, Time, And Safe Re-runs

Durability is an append-only log plus a snapshot
Time and ordering are plural and uncertain
Design every pipeline to be safe to re-run

Family E, Storage As A Substrate

A table is metadata over immutable files
Compose interchangeable layers through open standards

Family F, Operating Under Load And Trust

A system that cannot say no will fail
Data is a product with a contract

Family G, Policy And Economics

Enforce policy at the chokepoint
Cost is an architecture decision

Family H, Data For AI

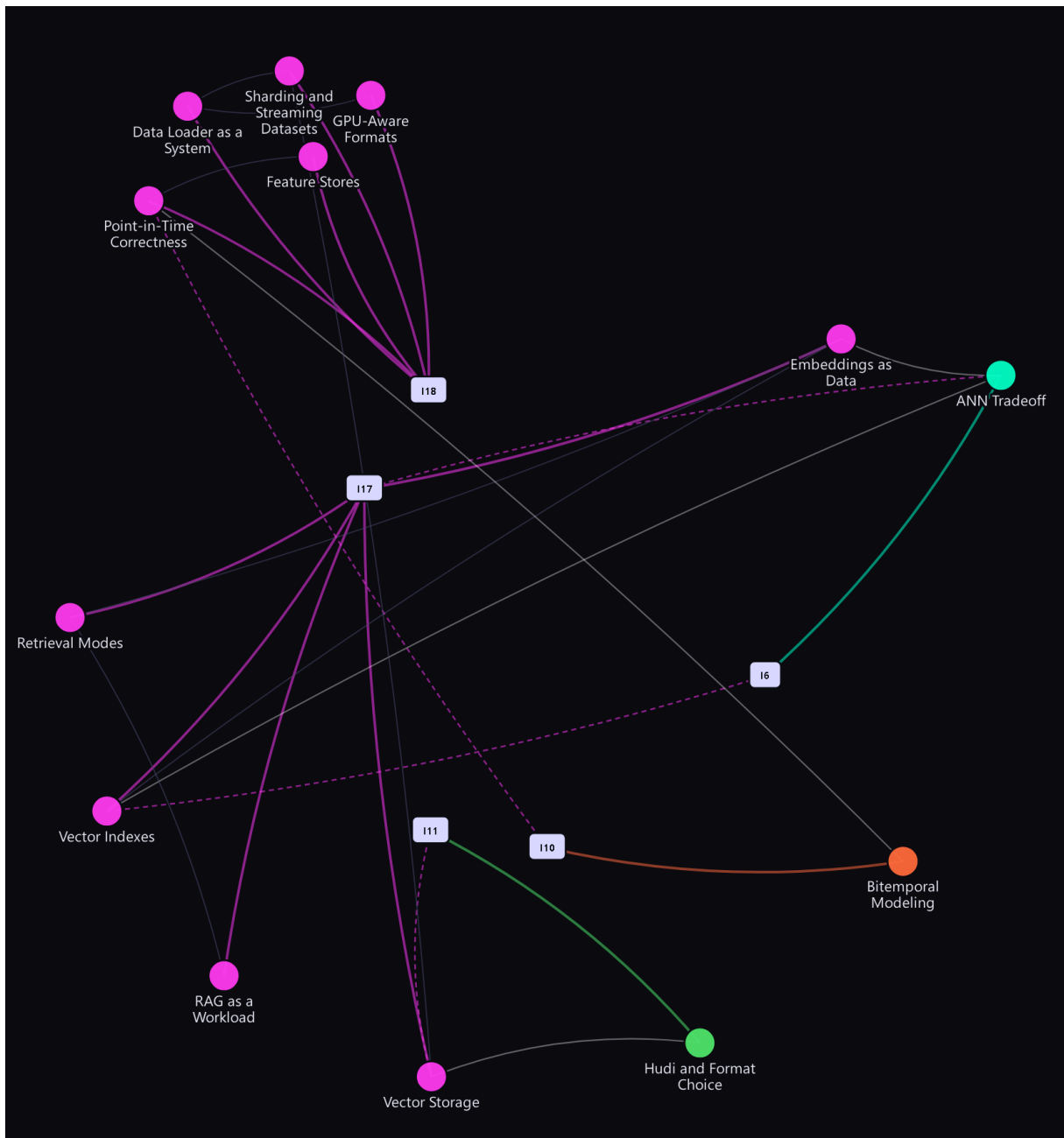
Meaning becomes geometry
Feed the accelerator

There.
You may now applaud.

Vogon Poetry - Interactive Explorer

A static figure cannot convey the dependency structure that is the central claim of this work; the interactive form lets the reader trace those dependencies directly. The viewer and the underlying data are available online at [**Vogon Poetry on GitHub**](<https://github.com/shauryashaurya/vogon-poetry>), and readers are invited to explore the concepts there. The 80 concepts and their typed relationships are published as an interactive concept graph that accompanies this paper. The viewer renders the 8 families, 18 instincts, and their concept nodes as a force-directed graph in the browser, colored by family and clustered by instinct, with the cross-family edges that encode where one invariant constrains another drawn explicitly. Readers can switch between layouts, filter by family or link type, isolate a single instinct and expand its concepts into readable cards, and search for individual nodes. Each node exposes its definition, a worked example on the canonical schema, and its links to related concepts.





Focussed subset of concepts more frequently encountered when starting Agentic AI systems

[...cont'd on next page]

Family A. Mechanical Sympathy

Instinct 1. Physical form dominates performance.

Logical data types are an abstraction. Every value has a physical byte layout, and that layout, not the logical type, sets the ceiling on scan and compute throughput.

Row vs Columnar

row-vs-columnar

- Row layout stores all fields of a record together; *columnar layout* stores each field contiguously.
- Point lookups favor rows; scans and aggregations favor columns.

Analogy. A row store is a filing cabinet with one folder per customer. A column store is a spreadsheet where each attribute is its own strip you can sum without opening folders.

Related

- *cache locality, SIMD, OLTP vs OLAP, Parquet, projection pushdown*
- Ideas: *bits and bytes primitives, fixed-width vs variable-width types, null representation as validity bitmaps*

Example / Illustrative Code

-- columnar engine reads only the 2 columns it needs, not whole rows

```
SELECT asset_class, sum(market_value) FROM positions GROUP BY asset_class;
```

Encoding Schemes

encoding-schemes

- *Dictionary encoding, run-length encoding, bit-packing, and delta encoding* shrink columnar data by exploiting repetition and ranges.
- They run before any general compressor, so the compressor then has less to do.

Analogy. Instead of writing "GBP" a million times, write it once in a legend and store small ticket numbers that point at it.

Related

- *compression codecs, row group hierarchy, Apache Arrow*
- Ideas: *run-length encoding, bit-packing, delta encoding*

Example / Illustrative Code

trades.currency has few distinct values, so a dictionary stores small codes

pyarrow applies dictionary encoding and bit-packing automatically on Parquet write

Compression Codecs

compression-codecs

- *Snappy*, *LZ4*, *ZSTD*, and *gzip* trade CPU cycles for fewer bytes moved.
- The right pick depends on whether the workload is I/O bound or CPU bound.

Analogy. Vacuum-packing luggage. Tighter packing saves cargo space but costs time at both ends.

Related

- *encoding schemes*, the CPU versus I/O tradeoff, *page cache*, *sequential I/O*

Example / Illustrative Code

ZSTD for cold, rarely read data; Snappy for hot, frequently scanned data

```
df.write.option("compression", "zstd").parquet(path)
```

Apache Arrow

apache-arrow

- *Apache Arrow* is a language-agnostic in-memory *columnar layout* standard.
- A shared format lets engines and libraries pass data without re-serializing it.

Analogy. A common shipping pallet size. Any forklift in any warehouse can move it without repacking.

Related

- *zero-copy*, *Arrow Flight*, *Parquet*, *SerDe* cost, *Polars* and *pandas* interop
- Ideas: storage layout versus in-memory layout, the *file* and *row group* and *page* hierarchy

Example / Illustrative Code

```
import pyarrow as pa
```

```
t = pa.table({"trade_id": [1, 2], "price": [99.5, 100.0]}) # shareable, no recopy
```

Instinct 2. Movement is the cost.

CPU cycles are cheap relative to moving bytes across copies, processes, disks, and networks. Expert design minimizes and amortizes movement.

Copies and SerDe

copies-and-serde

- Every boundary at the operating system, process, and runtime level can force a buffer copy.

- Converting between formats, called *SerDe*, is often the largest single cost in a pipeline.

Analogy. Each time a parcel changes courier it gets unpacked and repacked. The repacking, not the driving, eats the day.

Related

- *zero-copy, Apache Arrow, Protobuf and gRPC, deserialization*
- Ideas: buffer copies at the operating-system, process, and runtime levels

Example / Illustrative Code

avoid: `df.toPandas()` then rebuilding a Spark frame forces a full SerDe round trip

prefer: stay in one engine, or hand off through Arrow

Zero-Copy

zero-copy

- *mmap*, shared memory, and *Arrow IPC* let two consumers read the same bytes with no copy.
- The fastest copy is the one you never make.

Analogy. Two people reading one whiteboard instead of each transcribing it into a notebook.

Related

- *Apache Arrow, page cache, Arrow Flight, mmap*

Example / Illustrative Code

```
import pyarrow as pa
```

```
buf = pa.memory_map("trades.arrow") # read without copying into the heap
```

The Shuffle

the-shuffle

- A *shuffle* redistributes rows across the cluster by key so all rows with the same key meet.
- It writes to disk, transfers over the network, and reads back, which makes it the most expensive distributed operation.

Analogy. Telling a stadium crowd to re-seat themselves by birth month. Everyone moves at once and the aisles jam.

Related

- wide dependencies, *partitioning, distributed joins, skew, spilling*
- Links to Instinct 7 (Placement decides what is cheap)

Example / Illustrative Code

```
df.groupBy("customer_id").sum("amount") # triggers a shuffle on customer_id
```

Columnar Transport
columnar-transport

- *Arrow Flight* and *Flight SQL* move columnar batches over the wire without row-by-row encoding.
- This removes *SerDe* from the network path.

Analogy. Shipping a pre-loaded container instead of handing over boxes one at a time at the dock.

Related

- *gRPC*, *Apache Arrow*, *RPC* versus streaming transport, *connection pooling*
- Ideas: *Thrift* and *Protobuf*, *pagination* and *batching*

Example / Illustrative Code

```
# a Flight SQL client receives Arrow batches directly, with no per-row parse
```

```
# client.execute("SELECT * FROM trades").read_all()
```

Instinct 3. Batch beats tuple-at-a-time.

Per-item overhead from virtual calls, bounds checks, and cache misses dominates when you process one value at a time. Processing batches amortizes that overhead and unlocks SIMD.

Volcano Model
volcano-model

- The classic *Volcano model* iterator pulls one tuple at a time through a tree of operators.
- It is simple and composable, but the per-tuple function-call overhead is large.

Analogy. Passing one brick at a time down a line of workers, each pausing to receive and hand off.

Related

- *vectorized execution*, *code generation*, *pipelining*
- Ideas: *pull-based execution*

Example / Illustrative Code

```
# conceptual: for row in child.next(): emit(predicate(row)) # one call per row
```

Vectorized Execution

vectorized-execution

- Operators process batches of values, for example 1024 at a time, so the CPU runs tight loops and *SIMD* instructions.
- This amortizes dispatch cost across the whole batch.

Analogy. Stamping a whole sheet of forms in one press rather than one form per pull of the lever.

Related

- *SIMD*, *cache locality*, *Apache Arrow*, *DuckDB* and *Polars*, *code generation*

Example / Illustrative Code

```
import polars as pl
```

```
pl.scan_parquet("trades.parquet").filter(pl.col("price") > 100).collect() # batched
```

Code Generation

code-generation

- Whole-stage *code generation* and *JIT* compile a query fragment into specialized machine code.
- It removes interpreter overhead and is used by *Photon*, *DuckDB*, and *Velox*.

Analogy. Casting a custom tool for one job instead of reaching for a general adjustable wrench on every turn.

Related

- *vectorized execution*, *Photon*, *Velox*, *DataFusion*
- Ideas: interpreted-vectorized versus *JIT*-generated execution, pipelining versus materialization

Example / Illustrative Code

```
# Spark fuses filter, project, and aggregate into one generated function
```

```
# inspect with df.explain(mode="codegen")
```

Family B. Do Less, and Prove It

Instinct 4. Declare what; let the planner choose how.

You declare intent as SQL or a DataFrame. The engine derives a logical plan, rewrites it, and chooses a physical plan from statistics. Separating what from how is what lets the same query get faster with no rewrite.

Three Plan Levels

three-plan-levels

- A query exists at three levels: the logical plan (what), the physical plan (how, with chosen operators), and the *DAG* of distributed tasks that actually runs.
- An expert can point to any one of the three on demand.

Analogy. A trip is a destination, then a chosen route and transport, then the turn-by-turn legs you drive.

Related

- *parsing* and *abstract syntax trees*, query optimization, *DAG*, *lazy evaluation*

Example / Illustrative Code

EXPLAIN

```
SELECT c.country, sum(p.amount)
FROM payments p JOIN customers c USING(customer_id)
GROUP BY 1;
```

Lazy vs Eager

lazy-vs-eager

- Lazy engines such as *Spark*, *Polars*, and *Dask* build the full plan before executing and optimize across the whole query.
- Eager engines such as *pandas* run each step at once and cannot reorder.

Analogy. A chef reading the whole recipe before starting versus shopping for each ingredient mid-cook.

Related

- *DAG*, query optimization, *pushdown*, three plan levels

Example / Illustrative Code

```
import polars as pl
q = pl.scan_parquet("payments.parquet").filter(pl.col("amount") > 1e6) # nothing runs

q.collect() # plan is optimized, then executed
```

Rule-Based Optimization

rule-based-optimization

- Mechanical rewrites that are always safe: *predicate pushdown*, *projection pushdown*, *constant folding*.
- They shrink the work before any cost is considered.

Analogy. Crossing items off a shopping list that you already have at home before driving to the store.

Related

- *pushdown*, the skipped-work instinct, *cost-based optimization*
- Ideas: *constant folding*

Example / Illustrative Code

WHERE amount > 1e6 is pushed into the Parquet scan; non-matching row groups are skipped

Cost-Based Optimization

cost-based-optimization

- Using table statistics such as row counts, *number of distinct values*, and *histograms* to choose join order, join algorithm, and physical operators by estimated cost.
- The optimizer is only as good as the statistics it is given.

Analogy. A navigation app picking a route from live traffic estimates rather than always taking the straightest line.

Related

- *statistics*, *distributed joins*, *broadcast* versus *shuffle*, *adaptive query execution*
- Ideas: *number of distinct values*, *histograms*, optimization criteria such as latency versus shuffle bytes

Example / Illustrative Code

ANALYZE TABLE trades COMPUTE STATISTICS FOR COLUMNS instrument_id; -- feeds the cost model

Adaptive Query Execution

adaptive-query-execution

- *Adaptive query execution* replans mid-flight using real runtime statistics.
- For example it switches a *sort-merge join* to a *broadcast join* once it sees the true build-side size.

Analogy. Re-routing your drive after you actually hit the jam, not just from the morning forecast.

Related

- *cost-based optimization, skew handling, broadcast versus shuffle, runtime filters*

Example / Illustrative Code

```
spark.conf.set("spark.sql.adaptive.enabled", "true") # coalesce partitions, fix skew at runtime
```

Instinct 5. The fastest work is skipped work.

I/O you never issue is free. Layouts and indexes that let the engine prove a chunk is irrelevant beat any amount of faster scanning.

Data Skipping data-skipping

- Min and max *zone maps* and *bloom filters* stored per file or block let the engine skip data that cannot match a predicate.
- This avoids I/O entirely rather than scanning faster.

Analogy. A library catalog telling you a wing holds no books in your subject, so you never walk in.

Related

- *predicate pushdown, partitioning, bloom filters, column indexes*
- Ideas: min and max *zone maps*, page-level statistics

Example / Illustrative Code

```
-- row-group min and max on trade_ts let WHERE trade_ts >= DATE '2025-01-01' skip old groups
```

Pushdown pushdown

- Pushing predicates, projections, aggregates, limits, and even joins down into the storage layer or remote source.
- Less data is read and transferred as a result.

Analogy. Asking the warehouse to ship only the red size-10 shoes instead of shipping everything and sorting at home.

Related

- *rule-based optimization, federation, data skipping*
- Ideas: *projection pushdown, aggregate pushdown, limit pushdown, top-k pushdown*

Example / Illustrative Code

```
spark.read.parquet(path).select("trade_id", "price").filter("venue = 'LSE'") # both pushed
```

Partitioning Strategies**partitioning-strategies**

- *Hash partitioning, range partitioning, and round-robin partitioning* decide how rows map to partitions.
- The right choice aligns physical placement with the query's filter and join keys.

Analogy. Filing invoices by month, which is range, versus by client initial, which is hash, depending on how you search them.

Related

- *bucketing, pruning, distributed joins, skew*
- Ideas: *round-robin partitioning*

Example / Illustrative Code

```
df.write.partitionBy("trade_date").parquet(path) # filters on trade_date prune directories
```

Bucketing and Clustering**bucketing-clustering**

- Persisting a partitioning or sort order to storage so future queries skip the *shuffle* or skip files.
- *Clustering* co-locates related rows physically.

Analogy. Pre-sorting the mailroom shelves so the daily delivery never has to be re-sorted.

Related

- *partitioning, shuffle avoidance, table maintenance*
- Ideas: *space-filling curves, Z-order, Hilbert curves*

Example / Illustrative Code

```
df.write.bucketBy(64, "customer_id").sortBy("customer_id").saveAsTable("payments_bucketed")
```

Instinct 6. Approximate on purpose.

Exact answers are often unaffordable and unnecessary. Bounded-error structures give orders-of-magnitude savings for cardinality, quantiles, and similarity.

Sketches**sketches**

- Compact probabilistic summaries: *HyperLogLog* for distinct counts, *t-digest* for quantiles, *Count-Min sketch* for frequencies.
- They use constant memory, are mergeable, and have bounded error.

Analogy. Estimating crowd size from one sampled section rather than counting every head.

Related

- *statistics, number of distinct values, streaming aggregation, learned indexes*
- Ideas: *Count-Min sketch, t-digest, reservoir sampling*

Example / Illustrative Code

```
SELECT approx_count_distinct(customer_id) FROM payments; -- HyperLogLog under the hood
```

ANN Tradeoff

ann-tradeoff

- *Approximate nearest neighbor* search trades a little recall for large latency gains.
- Every vector index exposes a recall, latency, and build-cost triangle.

Analogy. Asking a few well-connected locals for the nearest cafe instead of measuring the distance to every cafe in the city.

Related

- *vector indexes, HNSW, IVF, embeddings*
- Links to Instinct 17 (Meaning becomes geometry)

Example / Illustrative Code

```
# HNSW: a higher ef_search raises recall and latency together
```

```
# index.search(query_vec, k=10, ef_search=128)
```

Learned Indexes

learned-indexes

- Replacing a traditional index with a model that predicts the storage position of a key.
- It works when the key distribution is learnable.

Analogy. Guessing a word's page in a dictionary from its first letter instead of binary searching every time.

Related

- *B-trees, sketches, cost-based optimization*

Example / Illustrative Code

```
# model maps key -> approximate offset, then a small local search corrects the guess
```

Family C. Distribution

Instinct 7. Placement decides what is cheap.

In a distributed engine the cost of an operation is set by whether the data it needs is already co-located. Joins and aggregations are cheap when keys are aligned and expensive when they force a shuffle.

Narrow vs Wide Dependencies**narrow-vs-wide**

- Narrow dependencies such as map and filter need no data movement.
- Wide dependencies such as group-by and joins on unaligned keys require a *shuffle*, and that boundary defines stage boundaries.

Analogy. Narrow is each worker finishing their own pile. Wide is everyone stopping to swap piles by category.

Related

- *shuffle, partitioning, DAG, stages*

Example / Illustrative Code

```
df.filter("amount > 1e6")    # narrow, no shuffle
```

```
df.groupBy("account_id").count() # wide, shuffle
```

Distributed Joins**distributed-joins**

- *Broadcast join* ships a small table to every node; *shuffle hash join* repartitions both sides by key; *sort-merge join* sorts both sides.
- Size and skew pick the winner.

Analogy. Broadcast hands everyone a pocket reference. Shuffle re-seats both groups by key. Sort-merge lines both up in order and walks them together.

Related

- *broadcast* versus *shuffle*, *skew*, *partitioning*, *cost-based optimization*
- Ideas: *nested loop join*, *in-memory hash join*, *grace hash join*, *sort-merge join* with *external sort*

Example / Illustrative Code

```
from pyspark.sql.functions import broadcast

trades.join(broadcast(instruments), "instrument_id") # ship the small dimension everywhere
```

Broadcast vs Shuffle**broadcast-vs-shuffle**

- The deciding factors are the build-side size estimate, selectivity, and *skew*.
- A wrong estimate turns a cheap *broadcast join* into an out-of-memory failure or forces a needless *shuffle*.

Analogy. Deciding whether to mail everyone a copy or call a central meeting depends on how big the document is.

Related

- *cost-based optimization, adaptive query execution, statistics, skew*

Example / Illustrative Code

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10 * 1024 * 1024) # 10 MB cutoff
```

Skew and Salting**skew-and-salting**

- *Skew* is when a few keys hold most of the rows and overload one task.
- *Salting* splits hot keys across tasks, and *adaptive query execution* can detect and split skewed partitions.

Analogy. One checkout lane jammed because everyone buys the same item, so you open extra lanes just for that item.

Related

- *partitioning, shuffle, adaptive query execution, distributed joins*

Example / Illustrative Code

```
from pyspark.sql.functions import concat_ws, rand

# spread a hot customer_id across 8 sub-keys, join, then aggregate the salt away
df.withColumn("k_salt", concat_ws("_", "customer_id", (rand() * 8).cast("int")))
```

Spilling**spilling**

- When an operator exceeds its memory budget it spills to disk and continues, trading speed for completion.
- Spill-aware operators degrade gracefully instead of failing.

Analogy. Running out of desk space and stacking overflow papers on the floor rather than stopping work.

Related

- memory budgets, *external sort*, *grace hash join*, *backpressure*
- Ideas: *external hash join*, run generation and merge

Example / Illustrative Code

```
# external sort writes sorted runs to disk then merges them
```

```
# spill bytes show up in the Spark UI stage metrics
```

Family D. State, Time, and Safe Re-runs

Instinct 8. Durability is an append-only log plus a snapshot.

Databases do not overwrite data in place and hope for the best. They append every change to a *write-ahead log* for durability and keep a *snapshot* or index for fast reads. Recovery replays the log onto the last snapshot.

Write-Ahead Log

write-ahead-log

- Every change is appended to a *write-ahead log* and flushed before the data pages change.
- A crash is recovered by replaying the log, which makes the log the foundational durability primitive.

Analogy. Writing your move in a notebook before you move the piece, so an interrupted game can resume exactly.

Related

- *fsync*, recovery, *LSM tree*, *MVCC*

Example / Illustrative Code

```
# append the change record, fsync, then mutate pages; replay the log on restart
```

```
# a payment insert is durable once its log record is flushed
```

B-Tree

b-tree

- The workhorse *OLTP* index: balanced and sorted, strong for point lookups and range scans, updated in place.
- It is the default index behind most relational engines.

Analogy. A library index with sorted tabs you binary-search instead of walking every shelf.

Related

- *B+ tree, OLTP, page, learned indexes*

Example / Illustrative Code

```
CREATE INDEX idx_accounts_id ON accounts(account_id); -- B-tree for point lookups and ranges
```

LSM Tree

lsm-tree

- Writes buffer in an in-memory *memtable*, flush to sorted *SSTables*, and merge through *compaction*.
- This design dominates write-heavy systems because writes are sequential.

Analogy. Jotting notes on sticky pads, then periodically filing them in sorted order.

Related

- *memtable, SSTable, compaction, copy-on-write versus merge-on-read*

Example / Illustrative Code

```
# writes land in a memtable, flush to immutable SSTables, compaction merges and sorts
```

```
# suits high-rate trade capture and surveillance event stores
```

MVCC

mvcc

- *Multi-version concurrency control* lets readers see a consistent *snapshot* while writers create new versions.
- Readers and writers do not block each other.

Analogy. Each reader gets a photocopy of the ledger as of the moment they started reading.

Related

- *snapshot isolation, time travel, copy-on-write*

Example / Illustrative Code

-- a long report reads a consistent snapshot while new trades keep arriving

SELECT count(*) FROM trades; -- sees the data as of statement start

Copy-on-Write vs Merge-on-Read

COW-VS-MOR

- *Copy-on-write* rewrites whole files on update, which is read-fast and write-heavy.
- *Merge-on-read* writes small deltas merged at read time, which is write-fast and read-heavier. The same tradeoff reappears in lakehouse formats.

Analogy. Re-typing the whole document on every edit versus keeping a list of edits applied only when someone reads it.

Related

- *deletion vectors, Hudi, Delta Lake, Iceberg*
- Links to Instinct 11 (A table is metadata over immutable files)

Example / Illustrative Code

copy-on-write rewrites the data file; merge-on-read appends a delta merged on read

revised reinsurance claims favor merge-on-read for cheap frequent updates

Instinct 9. Time and ordering are plural and uncertain.

There is no single notion of when. *Event time*, *ingestion time*, and *processing time* differ, ordering holds only within a partition, and completeness is something you estimate with *watermarks* rather than know for certain.

Event Time vs Processing Time

event-vs-processing-time

- *Event time* is when something happened, *ingestion time* is when it arrived, and *processing time* is when the engine handled it.
- Late and out-of-order data make the three diverge.

Analogy. A postcard carries the date it was written, the postmark date, and the day you finally read it.

Related

- *watermarks, windowing, late data*
- Reinsurance fits well: *loss_date* is event time, *report_date* is ingestion time

Example / Illustrative Code

-- claims can be reported years after the loss occurs

```
SELECT date_trunc('year', loss_date) AS yr, count(*) FROM claims GROUP BY 1;
```

Watermarks**watermarks**

- A *watermark* is the engine's estimate that no events older than time T will still arrive, which lets it close a window.
- Completeness is therefore a bet, not a certainty.

Analogy. Closing the ballot box at a deadline, accepting that a few late votes may be lost.

Related

- *late data, triggers, allowed lateness*

Example / Illustrative Code

```
events.withWatermark("trade_ts", "10 minutes").groupBy(window("trade_ts", "1 minute")).count()
```

Windowing**windowing**

- *Tumbling, sliding, and session* windows cut an unbounded stream into finite chunks to aggregate.
- The window type encodes the business question.

Analogy. Counting traffic per fixed hour, per rolling hour, or per gap-separated rush.

Related

- *watermarks, triggers, streaming joins*

Example / Illustrative Code

```
from pyspark.sql.functions import window
```

```
payments.groupBy(window("payment_ts", "1 minute")).sum("amount") # tumbling 1-minute windows
```

Ordering Guarantees**ordering-guarantees**

- Ordering holds within a partition but not across partitions, and global order costs coordination.
- The partition key decides what stays ordered.

Analogy. Each checkout line is ordered, but there is no single order across all the lines.

Related

- *partitioning, idempotency*
- Ideas: *wall-clock* versus *logical clocks* versus *hybrid logical clocks*

Example / Illustrative Code

Kafka preserves order within a partition keyed by `account_id`, not across partitions

choose the partition key so per-account events stay ordered

Instinct 10. Design every pipeline to be safe to re-run.

Failures, *backfills*, and replays are normal operations. A pipeline you cannot run twice is a liability. *Idempotency*, *determinism*, and explicit history make re-running boring and safe.

Idempotency and Determinism

idempotency-determinism

- *Idempotent* writes produce the same end state on re-run; *deterministic* transforms produce the same output for the same input.
- Together with *replayability* they make recovery safe.

Analogy. A switch set to on lands in the same state no matter how many times you push it toward on.

Related

- *exactly-once, backfills, upsert* and *MERGE*

Example / Illustrative Code

-- re-running this MERGE over replayed payments does not double count

MERGE INTO payments t USING staged s ON t.payment_id = s.payment_id

WHEN NOT MATCHED THEN INSERT *;

Change Data Capture

change-data-capture

- *Change data capture* turns a source database's changes into a stream by reading its log, using triggers, or polling queries.
- Log-based CDC has the lowest impact on the source.

Analogy. Subscribing to a newspaper's corrections column instead of re-reading the whole paper each day.

Related

- *Debezium, incremental view maintenance, streaming lakehouse*

Example / Illustrative Code

```
# log-based CDC reads the database write-ahead log and emits row changes
```

```
# Debezium streams accounts inserts, updates, and deletes into the lakehouse
```

Incremental View Maintenance

incremental-view-maintenance

- Updating a *materialized view* from only the changed rows instead of recomputing from scratch.
- Cost scales with the change size, not the table size.

Analogy. Updating a running total by adding the new receipt, not re-adding every receipt.

Related

- *change data capture, materialized views, streaming joins*

Example / Illustrative Code

```
# maintain daily position totals from only the new trades
```

```
# new_total = prev_total + sum(changed_rows)
```

Slowly Changing Dimensions

slowly-changing-dimensions

- *Slowly changing dimensions* track attribute changes over time: Type 1 overwrites, Type 2 adds a versioned row, Type 3 keeps a prior-value column.
- The type chosen decides how much history survives.

Analogy. Type 1 erases the old address, Type 2 keeps a dated address history, Type 3 keeps only the previous address.

Related

- *bitemporal modeling, MERGE, time travel*
- Banking: a history of *customers.risk_rating*

Example / Illustrative Code

-- SCD Type 2: close the old risk_rating row and insert a new versioned row

-- keeps full history of customers.risk_rating with valid_from and valid_to

Bitemporal Modeling

bitemporal-modeling

- *Bitemporal modeling* tracks *valid time*, when a fact was true in the world, and *system time*, when the system recorded it.
- This enables true audit and as-of reconstruction.

Analogy. A ledger that records both when an event occurred and when you wrote it down, so you can ask what you believed last Tuesday.

Related

- *slowly changing dimensions, MVCC, time travel*
- Reinsurance fits strongly: claim reserve revisions over time

Example / Illustrative Code

-- ask what we believed about a claim as of a past date

```
SELECT * FROM claims_bitemporal
```

```
WHERE claim_id = 42 AND valid_time <= DATE '2025-03-01' AND system_time <= DATE '2025-03-15';
```

Family E. Storage as a Substrate

Instinct 11. A table is metadata over immutable files.

On *object storage* there is no in-place update and no atomic rename. A table is a *metadata layer* that points at a set of immutable files, and a commit swaps metadata rather than data. This is the entire premise of the *lakehouse*.

Object Storage Semantics

object-storage-semantics

- Object stores such as *S3*, *GCS*, and *ADLS* meter listing, lack atomic rename, and were historically eventually consistent.
- Files are immutable, so you replace rather than edit.

Analogy. A mail depot where you can drop and fetch parcels but never reach in to edit one.

Related

- *lakehouse, multipart upload, throttling*
- Ideas: listing cost, eventual-consistency history

Example / Illustrative Code

no in-place edit and no atomic rename; write a new object and swap a pointer

listing a prefix with millions of objects is slow and metered

Lakehouse Premise

lakehouse-premise

- A *metadata layer* brings warehouse semantics such as *ACID*, schema, and *time travel* to object-storage economics.
- It does so by separating the metadata layer from the file layer.

Analogy. A card catalog that turns a warehouse of unmarked boxes into a queryable library.

Related

- *Iceberg, Delta Lake, Hudi, catalogs*

Example / Illustrative Code

a metadata layer over immutable Parquet files provides ACID, schema, and time travel

Iceberg Internals

iceberg-internals

- *Iceberg* defines a table with a metadata tree of *snapshots* and *manifests*, supports *hidden partitioning* and safe schema evolution.
- v2 and v3 add row-level and *equality deletes* and *deletion vectors*.

Analogy. A versioned table of contents that lists exactly which files make up the table right now.

Related

- *REST catalog, partition evolution, copy-on-write versus merge-on-read*

Example / Illustrative Code

-- read the table as of an older snapshot

SELECT * FROM trades FOR SYSTEM_VERSION AS OF 8273465;

Delta Internals

delta-internals

- *Delta Lake* uses a *transaction log* of JSON commits plus Parquet *checkpoints*, and *deletion vectors* for merge-on-read deletes.
- *UniForm* exposes Iceberg-compatible metadata for the same files.

Analogy. A commit log like version control, periodically snapshotted so you never replay from the beginning.

Related

- *change data feed*, *column mapping*, *deletion vectors*

Example / Illustrative Code

`_delta_log` holds JSON commits plus periodic Parquet checkpoints

deletion vectors mark deleted rows without rewriting the data file

Hudi and Format Choice

hudi-and-format-choice

- *Hudi* offers *copy-on-write* and *merge-on-read* tables with a *timeline* of instants and *record-level indexes*.
- Choosing among *Iceberg*, *Delta Lake*, *Hudi*, and *Lance* is a function of write pattern, ecosystem, and AI needs.

Analogy. Picking a filing system by how often you revise documents versus how often you read them.

Related

- *timeline*, *record-level index*, *Lance*
- Ideas: metadata architecture, write path, read path, concurrency models, CDC support, vector support, catalog compatibility

Example / Illustrative Code

copy-on-write for read-heavy tables, merge-on-read for write-heavy tables

pick Iceberg, Delta, Hudi, or Lance by write pattern, ecosystem, and AI needs

Table Maintenance

table-maintenance

- *Compaction* merges small files, *vacuum* and *snapshot expiration* remove dead files, and *Z-order* clustering improves skipping.
- Unmaintained lakehouse tables degrade in both cost and speed.

Analogy. Defragmenting the cabinet and shredding old drafts so it stays fast and lean.

Related

- *small-file problem, Z-order, snapshot expiration*

Example / Illustrative Code

OPTIMIZE trades ZORDER BY (trade_ts); -- compact small files and cluster for skipping

VACUUM trades RETAIN 168 HOURS; -- expire dead files

Instinct 12. Compose interchangeable layers through open standards.

Modern systems are not monoliths. Storage, *catalog*, execution engine, and query frontend are separate layers joined by open formats and plan intermediate representations, so each can be swapped independently.

Disaggregated Storage and Compute disaggregated-storage-compute

- Separating storage from compute lets each scale on its own and lets compute be ephemeral.
- It is the defining pattern of every modern cloud data system.

Analogy. Renting meeting rooms by the hour while your files live permanently in a vault.

Related

- *object storage, autoscaling, cost*
- Links to Instinct 16 (Cost is an architecture decision)

Example / Illustrative Code

data lives in object storage; compute clusters spin up, read, and shut down

Pluggable Backends pluggable-backends

- *Velox, DataFusion, Photon*, and *Gluten* are drop-in *vectorized execution* layers reused across engines.
- They let several products share one high-performance core.

Analogy. A standard engine block that several car brands bolt their own body onto.

Related

- *vectorized execution, code generation, Substrait*

Example / Illustrative Code

Velox, DataFusion, Photon, and Gluten provide a reusable vectorized execution layer

Engine Architectures engine-architectures

- MPP engines like *Trino* and *Redshift*, embedded engines like *DuckDB* and *Polars*, and scale-out engines like *Spark* and *ClickHouse* suit different sizes and latencies.
- The right class depends on data size and concurrency, not fashion.

Analogy. A fleet, a hatchback, and a freight train for different journeys.

Related

- *single-node renaissance, federation*

Example / Illustrative Code

MPP: Trino, Redshift | embedded: DuckDB, Polars | scale-out: Spark, ClickHouse

Substrait substrait

- *Substrait* is a cross-engine logical plan intermediate representation.
- One frontend can target many backends, and one backend can serve many frontends.

Analogy. A shared blueprint format that any builder can read and construct from.

Related

- three plan levels, *pluggable backends, federation*
- Links to Instinct 4 (Declare what; let the planner choose how)

Example / Illustrative Code

one logical plan IR; a Polars or Spark frontend can target many execution backends

Federation federation

- *Federation* runs one query across heterogeneous engines and sources, pushing work to each.
- It avoids copying data just to query it.

Analogy. A general contractor coordinating specialist trades, each doing its part on site.

Related

- *pushdown, query routing, composable data systems*

- Ideas: *single-node renaissance*, *WASM* execution

Example / Illustrative Code

-- one query joins a Postgres table and a Parquet lake table, pushing filters to each

```
SELECT * FROM postgres.customers c JOIN lake.payments p USING(customer_id);
```

Family F. Operating Under Load and Trust

Instinct 13. A system that cannot say no will fail.

Unbounded acceptance leads to collapse. Memory budgets, *backpressure*, *admission control*, and scheduling let a system slow or shed work and degrade predictably instead of crashing.

Memory Budgets**memory-budgets**

- Per-operator memory accounting decides whether a query *spills* or fails.
- *On-heap* versus *off-heap* placement changes garbage-collection behavior under load.

Analogy. Each department gets a budget; overspending triggers a spill to a cheaper option, not bankruptcy.

Related

- *spilling*, spill-aware operators, *off-heap*
- Ideas: *arena allocators* and *bump allocators*, *garbage collection*

Example / Illustrative Code

```
spark.conf.set("spark.memory.fraction", "0.6") # operator budget; overflow spills, not crashes
```

Backpressure and Admission Control**backpressure-admission**

- *Backpressure* slows producers when consumers fall behind; *admission control* queues or rejects new work to protect SLAs.
- Both bound the work in flight.

Analogy. A busy restaurant pacing the kitchen and pausing the waitlist rather than seating everyone at once.

Related

- streaming, concurrency control, *schedulers*

Example / Illustrative Code

slow producers when the sink lags; queue or reject new queries to protect SLAs

Schedulers and Autoscaling schedulers-autoscaling

- *YARN* and *Kubernetes* allocate resources; *autoscaling* adds or removes capacity reactively or predictively.
- The policy trades dollar cost against latency.

Analogy. Calling in more staff when the queue grows and sending them home when it shrinks.

Related

- *Spark on Kubernetes*, *dynamic allocation*, cost
- Ideas: reactive versus predictive scaling

Example / Illustrative Code

Kubernetes allocates executors; dynamic allocation adds and removes them with load

Multi-Tenancy and Isolation multi-tenancy-isolation

- *Resource queues*, namespaces, and concurrency limits stop one tenant's heavy job from starving others.
- This is the cure for the *noisy neighbor* problem.

Analogy. Separate lanes on a shared road so one truck does not block every car.

Related

- priority and fairness, *admission control*, *schedulers*

Example / Illustrative Code

separate resource queues per team stop one heavy job from starving the others

Instinct 14. Data is a product with a contract.

Datasets are products with consumers. Treat them like code: a declared schema, a compatibility policy, *data lineage*, tests, and an SLA. Verify, do not assume.

Data Lineage data-lineage

- Dataset-level and column-level *data lineage* records where data came from and what it feeds.

- It enables impact analysis and faster debugging.

Analogy. A supply-chain label tracing every ingredient back to its farm.

Related

- *OpenLineage, Marquez, data contracts*

Example / Illustrative Code

```
# emit OpenLineage events so each dataset records its inputs and consumers
```

```
# column-level lineage shows report.total derives from payments.amount
```

Schema Evolution

schema-evolution

- Forward, backward, and full compatibility rules govern safe schema change.
- The wrong change silently breaks consumers downstream.

Analogy. Updating a form so that both old and new printed copies remain readable.

Related

- *data contracts, Avro and Parquet, Iceberg evolution*

Example / Illustrative Code

```
ALTER TABLE customers ADD COLUMN segment STRING; -- a nullable add is backward compatible
```

Data Contracts

data-contracts

- A *data contract* is a producer-consumer agreement with an explicit schema, semantics, and SLA.
- It is enforced in CI so a breaking change fails the build.

Analogy. A delivery contract specifying size, timing, and quality, signed by both sides.

Related

- *schema evolution, data quality, data lineage*

Example / Illustrative Code

```
# the producer of payments commits to a schema, freshness SLA, and semantics; CI checks it
```

Data Quality

data-quality

- Frameworks like *Great Expectations*, *Soda*, and *dbt tests* assert row counts, ranges, and uniqueness.
- They act as a gate that stops bad data from propagating.

Analogy. Quality-control sampling on a production line that halts the belt on a defect.

Related

- *testing*, *data contracts*, observability

Example / Illustrative Code

```
# Great Expectations: assert amount is non-negative and customer_id is present  
  
expect_column_values_to_be_between("amount", min_value=0, max_value=None)
```

Testing for Data

testing-for-data

- Unit tests on pure transforms, *property-based testing*, *snapshot testing*, and *data diffing* catch regressions before release.
- Pipelines become testable when transforms are pure functions.

Analogy. A test suite plus a before-and-after photo comparison for every change.

Related

- *CI* for SQL, dbt, and Spark, *synthetic data*, *data quality*

Example / Illustrative Code

```
# unit test a pure transform on a tiny banking fixture  
  
assert normalize_currency("gbp") == "GBP"
```

Family G. Policy and Economics

Instinct 15. Enforce policy at the chokepoint.

Security enforced in every application drifts and leaks. Enforce policy once at the layer everyone must pass, the *catalog* or engine, so it is consistent and auditable.

Catalog as Policy

catalog-as-policy

- The *catalog* above the engines is the natural place to enforce access, since every engine consults it.
- Policy lives in one place rather than in each application.

Analogy. One staffed gate to the building instead of a separate lock on every interior door.

Related

- *row-level security* and *column-level security*, *masking*, lakehouse catalogs

Example / Illustrative Code

```
GRANT SELECT ON payments TO ROLE analyst_emea; -- enforced at the catalog, every engine obeys
```

Row and Column Security

row-column-security

- *Row-level security* and *column-level security* restrict which rows and columns a principal can see.
- Enforcement sits at the catalog or engine layer.

Analogy. A redacted document where your clearance decides which lines and pages appear.

Related

- *masking*, policy enforcement, *data contracts*
- Banking: analysts see only their region's accounts

Example / Illustrative Code

```
-- analysts see only rows for their own branches
```

```
CREATE ROW FILTER region_filter ON accounts AS (branch_id IN current_user_branches());
```

Masking and Tokenization

masking-tokenization

- *Masking* irreversibly obscures a value; *tokenization* swaps it for a reversible token held in a vault.
- Both protect sensitive banking fields such as PII and card numbers.

Analogy. Blacking out a number versus replacing it with a claim-check stub you can later redeem.

Related

- *encryption, differential privacy, guardrails*

Example / Illustrative Code

masking blanks the card number to XXXX; tokenization swaps it for a reversible vault token

Encryption encryption

- *Encryption at rest and in transit with envelope encryption and bring-your-own-key, plus key rotation.*
- It protects data even when storage or network is compromised.

Analogy. A locked container plus a master key you control and can change at will.

Related

- *bring-your-own-key, data residency, catalog policy*
- Ideas: *differential privacy, data residency*

Example / Illustrative Code

envelope encryption: a data key encrypts data, a master key (BYOK) encrypts the data key

Instinct 16. Cost is an architecture decision.

The cloud bill is a function of design, not an afterthought. Bytes scanned, bytes moved across regions, *storage tiering*, and the compute purchasing model are all chosen at architecture time.

Cost Units cost-units

- Reason in dollars per TB scanned, per query, and per GB stored as first-class design constraints.
- These units, not finance trivia, drive layout and pruning choices.

Analogy. Pricing a journey by fuel per mile before choosing the route.

Related

- *pushdown, partitioning, serverless economics*

Example / Illustrative Code

-- a serverless engine bills by bytes scanned; pruning columns and partitions cuts the bill

```
SELECT trade_id, price FROM trades WHERE trade_date = DATE '2025-06-01';
```

Storage Tiering

storage-tiering

- Hot, warm, cold, and archive tiers trade retrieval speed for storage price.
- *Lifecycle policies* move data between tiers automatically.

Analogy. Desk drawer, filing cabinet, basement, and off-site vault for how often you reach for it.

Related

- *lifecycle policy, egress, cost units*
- Reinsurance: old claims age into archive storage

Example / Illustrative Code

lifecycle policy moves claims older than 2 years from hot to archive storage

Compute Purchasing

compute-purchasing

- *Spot, on-demand, and reserved* capacity trade price against reliability and commitment.
- The mix should match how interruptible each workload is.

Analogy. Standby tickets, walk-up fares, and a season pass for different travel needs.

Related

- *autoscaling, schedulers*, disaggregation

Example / Illustrative Code

spot for fault-tolerant batch, reserved for steady baseline, on-demand for spikes

Cache vs Recompute

cache-vs-recompute

- Caching trades storage and staleness risk for repeated-query speed.
- Sometimes recomputing is cheaper than maintaining a cache.

Analogy. Keeping leftovers versus cooking fresh, depending on how often you eat the dish.

Related

- *materialized views, result caching, cost units*
- Ideas: *egress* and replication costs

Example / Illustrative Code

cache a daily aggregate if it is queried often; recompute if rarely read or cheap to derive

Family H. Data for AI

Instinct 17. Meaning becomes geometry.

Embeddings map meaning into vectors so that semantic similarity becomes distance. Retrieval becomes a geometry problem, and storage and indexing choices follow from that.

Embeddings as Data

embeddings-as-data

- An *embedding* is a learned dense vector that represents an item's meaning.
- It is a first-class data type whose access pattern is similarity, not equality.

Analogy. Placing every document on a map so that related ones sit near each other.

Related

- *vector indexes, quantization, retrieval*
- Ideas: embedding generation pipelines, *product quantization* and *binary quantization*

Example / Illustrative Code

an embedding is a dense vector; store it next to the row it represents

```
vec = embed("counterparty due diligence note") # shape (1024,)
```

Vector Indexes

vector-indexes

- *HNSW* is a navigable graph, *IVF* and *IVF-PQ* partition then search, and *DiskANN* and *ScaNN* serve billion-scale corpora from disk.
- Each exposes the recall, latency, and build-cost triangle.

Analogy. Neighborhood maps and shortcuts so you find nearby points without checking the whole city.

Related

- *ANN tradeoff, quantization*
- Links to Instinct 6 (Approximate on purpose)

Example / Illustrative Code

HNSW: graph of nearest neighbors | IVF and IVF-PQ: cluster then search | DiskANN: on disk

Vector Storage

vector-storage

- Dedicated stores such as *Pinecone*, *Weaviate*, *Milvus*, and *Qdrant*, extensions such as *pgvector* and *Elasticsearch*, and *Lance* and *LanceDB* keep vectors near the data.
- The choice trades operational simplicity against unification with analytical columns.

Analogy. A specialty library wing versus adding a vector shelf to the existing library.

Related

- *Lance*, lakehouse plus vector unification, *metadata filtering*
- Links to Instinct 11 (A table is metadata over immutable files)

Example / Illustrative Code

Lance stores vectors and columns together; query by similarity

```
tbl.search(query_vec).limit(10).to_arrow()
```

Retrieval Modes

retrieval-modes

- *Sparse retrieval* with *BM25* over an *inverted index* matches terms; *dense retrieval* matches *embeddings*; *hybrid search* blends both with a *reranker*.
- The mix is chosen for recall and precision needs.

Analogy. Keyword search, meaning search, and a judge who blends the two shortlists.

Related

- *reranker*, *hybrid search*, RAG
- Ideas: *cross-encoder* and LLM rerankers

Example / Illustrative Code

sparse: BM25 over an inverted index | dense: embedding similarity | hybrid: blend + rerank

RAG as a Workload

rag-as-workload

- *RAG* is a data pipeline: parse and *chunk* documents, index them, filter by metadata, keep fresh, and evaluate.
- It is data engineering, not a model trick.

Analogy. Running a research desk that files sources, retrieves the right ones on demand, and audits its hit rate.

Related

- *chunking*, freshness and incremental indexing, evaluation such as *recall at k* and *MRR*
- Ideas: document parsing, hierarchical chunking

Example / Illustrative Code

parse -> chunk -> embed -> index -> retrieve with metadata filter -> evaluate recall at k

Instinct 18. Feed the accelerator.

A GPU starves unless the data pipeline keeps up; the bottleneck is bandwidth, not compute. Training-data systems exist to keep accelerators fed with correct, reproducible data.

Feature Stores**feature-stores**

- A *feature store* serves the same feature definitions online at low latency and offline in batch.
- This avoids *online-offline skew* between training and serving.

Analogy. One recipe used in both the test kitchen and the restaurant so the dish never differs.

Related

- *point-in-time correctness*, *online-offline skew*
- Banking: fraud features computed identically at scoring and training time

Example / Illustrative Code

define a fraud feature once; serve it online for scoring and offline for training

Point-in-Time Correctness**point-in-time-correctness**

- Assemble training data using only information available as of the label time.
- This avoids *label leakage* that inflates offline metrics.

Analogy. Grading a forecast using only what was known before the event, never after.

Related

- *bitemporal modeling*, *label leakage*, *feature stores*
- Links to Instinct 10 (Design every pipeline to be safe to re-run)

Example / Illustrative Code

-- as-of join: attach only feature values known before the label timestamp

```
SELECT * FROM labels l ASOF JOIN features f ON f.entity = l.entity AND f.ts <= l.label_ts;
```

Data Loader as a System

data-loader-as-system

- The data loader is a CPU-to-GPU pipeline with workers, prefetch queues, and shared memory.
- It must hide I/O behind compute or the accelerator stalls.

Analogy. A pit crew handing over the next tire before the car stops, so the engine never waits.

Related

- *sharding, streaming datasets, GPUDirect Storage*
- Ideas: *PyTorch DataLoader* and *tf.data* internals

Example / Illustrative Code

```
# overlap I/O with compute so the GPU never waits
```

```
DataLoader(ds, num_workers=8, prefetch_factor=4, pin_memory=True)
```

Sharding and Streaming Datasets

sharding-streaming-datasets

- File-level *sharding* and streaming formats such as *WebDataset*, *MosaicML Streaming*, and *Lance* stream training data from object storage.
- They avoid hot spots across many GPUs.

Analogy. Dealing cards evenly from several decks so no single dealer is overworked.

Related

- *random access* versus *sequential access*, distributed training, *Lance*
- Ideas: *DDP*, *FSDP*, and *ZeRO* data views, checkpointing data state

Example / Illustrative Code

```
# shard files evenly and stream from object storage with WebDataset, Mosaic, or Lance
```

GPU-Aware Formats

gpu-aware-formats

- *Random access* on *Parquet* stalls training, so *Lance* and *tfrecord* shards exist for sequential and sharded reads.
- *GPUDirect Storage* loads data directly into device memory.

Analogy. A conveyor that drops parts straight onto the assembly arm instead of routing through a stockroom.

Related

- *random access* versus *sequential access*, training data formats, data loader
- Ideas: *GPUDirect Storage*, zero-copy into device memory

Example / Illustrative Code

random reads on Parquet stall training; Lance and tfrecord shards plus GPUDirect load fast

Canonical Schema used in examples

Banking

- **customers**(customer_id, name, country, risk_rating, onboarded_ts)
- **accounts**(account_id, customer_id, account_type, currency, branch_id, opened_ts)
- **instruments**(instrument_id, symbol, asset_class, currency)
- **trades**(trade_id, account_id, instrument_id, side, quantity, price, trade_ts, venue)
- **payments**(payment_id, account_id, counterparty_id, amount, currency, payment_ts, channel)
- **positions**(account_id, instrument_id, as_of_date, quantity, market_value)

Reinsurance

- **cedents**(cedent_id, name, country, rating)
 - **treaties**(treaty_id, cedent_id, line_of_business, treaty_type, inception_date, expiry_date)
 - **policies**(policy_id, treaty_id, insured_id, sum_insured, premium)
 - **claims**(claim_id, treaty_id, loss_date, report_date, paid_amount, reserve_amount, status)
-

Summary

This paper presents a structured map of the mental models that govern data systems in an AI-centric stack, where models commoditize and the data layer sets the correctness, latency, and unit cost of AI outputs. The claim is that an expert in the field must be able to think in this vocabulary instead of thinking in terms of tools, to effectively mine value in the evolving ecosystem.

The author does not introduce new systems or benchmarks. Instead this paper is a synthesis and a representation: the invariants/concepts, their instantiation, and the graph that encodes their interdependencies, intended as a basis for instruction and for reasoning about data architectures that prove essential in enterprise class AI-driven value chains.

References

- Vagon Poetry on GitHub: <https://github.com/shauryashaurya/vogon-poetry>